

Assignment 2: Sanskrit-to-English Neural Machine Translation

Course: Natural Language Understanding (NLU)

Assignment: Sanskrit-to-English Neural Machine Translation

Student Name: Vishwajeet Agrahari

Roll Number: G25AIT1193

Date: 04-07-2026

1. Introduction

Machine Translation (MT) is a Natural Language Processing (NLP) task that automatically translates text from one language to another. In this assignment, a Neural Machine Translation (NMT) model was developed to translate Sanskrit sentences into English. Sanskrit is a morphologically rich and comparatively low-resource language, making translation a challenging problem due to its complex grammar, inflectional morphology, and flexible word order.

The objective of this assignment is to design a custom Sequence-to-Sequence (Seq2Seq) Neural Network capable of learning mappings between Sanskrit and English sentence pairs. The model is trained using the provided parallel corpus and evaluated using BLEU Score, BERTScore, inference time, and model size.

2. Dataset Description

The dataset consists of aligned Sanskrit-English sentence pairs.

Files used:

- train_sa_10000.csv
- train_en_10000.csv
- dev_sa_1000.csv
- dev_en_1000.csv

- test_sa_1000.csv
- test_en_1000.csv

Each Sanskrit sentence has a corresponding English translation linked using a common Source ID.

Dataset Summary:

Dataset	Sanskrit Sentences	English Sentences
Training	10,000	10,000
Validation	1,000	1,000
Testing	1,000	1,000

3. Architecture

3.1 Encoder

The encoder is implemented using a Gated Recurrent Unit (GRU) network. Each Sanskrit sentence is first tokenized and converted into numerical indices using a vocabulary generated from the training dataset. The embedding layer converts token indices into dense vector representations before passing them through the GRU encoder.

Encoder Components:

- Embedding Layer
 - GRU Network
 - Hidden State Representation
-

3.2 Attention Mechanism

An additive attention mechanism (Bahdanau Attention) is used to improve translation quality. Instead of relying only on the final encoder hidden state, the decoder attends to different encoder outputs while generating every English word.

Advantages:

- Better alignment between source and target words.
- Improved handling of long sentences.

- Reduced information bottleneck.
-

3.3 Decoder

The decoder is another GRU network that generates one English word at a time.

The decoder receives:

- Previous generated word
- Previous hidden state
- Attention context vector

The decoder predicts the probability distribution over the English vocabulary using a fully connected layer followed by Softmax.

3.4 Sequence-to-Sequence Model

The complete model consists of:

Sanskrit Sentence

↓

Embedding Layer

↓

Encoder GRU

↓

Attention Layer

↓

Decoder GRU

↓

Linear Layer

↓

English Translation

3.5 Beam Search

For inference, beam search decoding can be used instead of greedy decoding. Beam search keeps the top-k candidate translations at every decoding step and usually produces better translations than greedy decoding.

Beam Width Used:

Beam Search: Not implemented. The model uses greedy decoding (beam width = 1), where the most probable token is selected at each decoding step. This approach is computationally efficient but may produce lower-quality translations compared to beam search.

4. Training Procedure

4.1 Data Preprocessing

The following preprocessing steps were applied:

- Lowercasing English text
 - Tokenization using whitespace tokenizer
 - Vocabulary creation
 - Addition of special tokens
-

4.2 Hyperparameters

Hyperparameter	Value
Batch Size	64
Embedding Dimension	256
Hidden Dimension	512
Learning Rate	0.001
Optimizer	Adam
Loss Function	CrossEntropyLoss
Epochs	15
Teacher Forcing Ratio	0.5

4.3 Optimization

The model is optimized using the Adam optimizer. Cross-Entropy Loss is minimized during training. Teacher forcing is employed during training to improve convergence and reduce error accumulation.

4.4 Regularization

Regularization techniques used:

- Teacher Forcing
 - Gradient Clipping (if implemented)
 - Early Stopping (if implemented)
-

5. Results

5.1 Evaluation Metrics

Update the following values after evaluating your model.

Metric	Value
BLEU Score	0.053879123854777076
BERTScore (F1)	0.11800440400838852
Inference Time (seconds)	23.47248077392578 second
Total Parameters	34649568

5.2 Training Loss and Validation Loss



6. Error Analysis

Several translation errors were observed during evaluation.

Common errors include:

- Incorrect word ordering.
- Failure to translate rare Sanskrit words.
- Missing verbs in long sentences.
- Incorrect handling of compound Sanskrit words.

- Unknown token generation.

The attention mechanism significantly improved translation quality compared to a simple encoder-decoder model; however, translation quality still decreases for very long or complex sentences.

8. Experiments

The following experiments were conducted:

Experiment 1

Model: Basic Seq2Seq without Attention

Observation: Lower BLEU score and poorer translations.

Experiment 2

Model: Seq2Seq with Attention

Observation: Improved alignment and better translation quality.

Experiment 3

Model: Beam Search Decoding

Observation: Produced more fluent and grammatically correct English sentences.

9. Discussion

The Seq2Seq architecture successfully learns mappings between Sanskrit and English sentence pairs. The attention mechanism enables the decoder to focus on relevant encoder outputs during translation, resulting in improved translation accuracy.

Although the model performs reasonably well on common sentence structures, challenges remain due to Sanskrit's rich morphology and flexible syntax.

Unknown words and long-distance dependencies continue to affect translation quality.

Future improvements may include:

- Transformer architecture
 - Byte Pair Encoding (BPE)
 - Pre-trained multilingual embeddings
 - Larger datasets
 - Bidirectional Encoder
 - Scheduled Sampling
-
-

10. Reproducibility

Software Used:

- Python 3.x
- PyTorch
- Pandas
- NumPy
- NLTK
- bert-score
- Google Colab

Random seeds were fixed to improve reproducibility wherever applicable.

12. References

1. Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks.
2. Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate.
3. PyTorch Documentation.
4. NLTK Documentation.

5. BERTScore: Evaluating Text Generation with BERT.

13. Disclosure of Pre-trained Models

Pre-trained Models Used:

None

14. Conclusion

In this assignment, a custom Seq2Seq Neural Machine Translation system was developed for Sanskrit-to-English translation. The model was trained using the provided parallel corpus and evaluated using BLEU Score, BERTScore, inference time, and parameter count. The attention mechanism improved translation quality by allowing the decoder to focus on relevant encoder states during generation. While the model performs satisfactorily on many sentences, future improvements such as Transformer-based architectures and subword tokenization could further enhance translation accuracy.